

# ИНСТИТУТ БИЗНЕСА И ДИЗАЙНА

ОТЧЁТ О ПРАКТИЧЕСКОЙ РАБОТЕ / № 01

## Базовый UI: компоненты, props и TypeScript

Разработка мобильных приложений · БИИФ-24

|       |                 |               |             |
|-------|-----------------|---------------|-------------|
| АВТОР | Телков Владимир | ПРЕПОДАВАТЕЛЬ | Саенко Я.Д. |
| ДАТА  | 16.09.2026      | ГОРОД         | Москва      |

### 01 / ПАСПОРТ ОТЧЁТА

#### Паспорт отчёта

| Поле                            | Значение                                                                                                                        |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Дисциплина                      | Разработка мобильных приложений                                                                                                 |
| Практическая работа             | № 01 — Базовый UI: компоненты, props и TypeScript                                                                               |
| Студент                         | Телков Владимир Сергеевич                                                                                                       |
| Группа                          | БИИФ-24                                                                                                                         |
| Преподаватель                   | Саенко Я.Д.                                                                                                                     |
| Дата                            | 16.09.2026                                                                                                                      |
| Связанный спринт                | Sprint 1 — статическая витрина справочника                                                                                      |
| Ссылка на реквест с изменениями | <a href="https://github.com/Leladatan/study-mobile-project/pull/2">https://github.com/Leladatan/study-mobile-project/pull/2</a> |
| Тег                             | sprint-1                                                                                                                        |

#### Цель работы

Сделать первый экран справочника из переиспользуемых типизированных компонентов.

#### Задачи

- описать тип основной сущности `Topic`;
- вынести данные стран в отдельный файл;
- сделать компонент карточки `TopicCard` с типизированными props;

- собрать экран каталога из базовых компонентов;
- проверить Fast Refresh и узнать, какой движок JavaScript используется;
- обновить README, backlog и CHANGELOG, открыть pull request.

---

## 02 / ВЫПОЛНЕНИЕ

### Что сделано

| Файл                                       | Содержание                         |
|--------------------------------------------|------------------------------------|
| <code>src/types/topic.ts</code>            | тип <code>Topic</code>             |
| <code>src/data/topics.ts</code>            | данные 12 стран                    |
| <code>src/components/TopicCard.tsx</code>  | карточка страны                    |
| <code>src/screens/CatalogScreen.tsx</code> | экран каталога                     |
| <code>src/lib/format.ts</code>             | форматирование населения           |
| <code>src/theme.ts</code>                  | цвета и отступы                    |
| <code>App.tsx</code>                       | показывает каталог вместо заглушки |

### Тип `Topic`

Сущность описана в `src/types/topic.ts`:

```
export type Topic = {
  id: string;
  title: string;
  description: string;
  imageUrl?: string;
  capital: string;
  regionId: RegionId;
  regionName: string;
  incomeLevel: IncomeLevel;
  population: number;
  populationYear: string;
  iso2: string;
};
```

`id`, `title` и `description` обязательные. `imageUrl` необязательный — это обозначает знак `?`. Остальные поля — данные о стране. `RegionId` и `IncomeLevel` — не просто строки, а списки допустимых значений, например `'EAS' | 'ECS'`

| .... Если в данных ошибиться в коде региона, TypeScript сразу покажет ошибку.

## Данные

Страны лежат в `src/data/topics.ts` в массиве `TOPICS: Topic[]`. Благодаря типу массива каждый объект проверяется на соответствие `Topic`. Вот пример — Канада:

```
{
  id: 'CAN',
  title: 'Canada',
  description:
    'Вторая страна мира по площади. Большая часть населения живёт вблизи границы с США.',
  capital: 'Ottawa',
  regionId: 'NAC',
  regionName: REGION_NAMES.NAC,
  incomeLevel: 'High income',
  population: 41651653,
  populationYear: '2025',
  iso2: 'CA',
},
```

`id` — код страны из трёх букв. Он уникальный и не меняется. У Канады специально нет `imageUrl`: на ней видно, как карточка выглядит без картинки.

## Компонент `TopicCard`

Контракт карточки:

```
export type TopicCardProps = {
  topic: Topic;
  onPress: (topic: Topic) => void;
  featured?: boolean;
};
```

- `topic` — данные страны;
- `onPress` — функция, которую карточка вызывает при нажатии;
- `featured` — необязательный проп, делает карточку крупнее. По умолчанию `false`.

`children` не используется: внутрь карточки ничего не вкладывается.

Если у страны есть флаг, показывается картинка, если нет — блок с кодом страны:

```
{topic.imageUrl ? (  
  <Image  
    source={{ uri: topic.imageUrl }}  
    style={[styles.flag, featured && styles.flagFeatured]}  
    resizeMode="cover"  
    accessibilityIgnoresInvertColors  
  />  
) : (  
  <View style={[styles.flag, styles.placeholder, featured &&  
    styles.flagFeatured]}>  
    <Text style={styles.placeholderText}>{topic.id}</Text>  
  </View>  
)}
```

Карточка не импортирует данные стран. Она получает объект через props и при нажатии вызывает `onPress(topic)`, а что делать дальше, решает экран.

## Экран каталога

Экран хранит выбранную страну в state:

```
const [selected, setSelected] = useState<Topic | null>(null);  
  
const handlePress = (topic: Topic) => {  
  setSelected(topic);  
};
```

И выводит карточки перебором массива:

```
{TOPICS.map((topic, index) => (  
  <TopicCard  
    key={topic.id}  
    topic={topic}  
    onPress={handlePress}  
    featured={index === 0}  
  />  
))}
```

Разметка карточки написана один раз, экран только передаёт в неё данные. Первая карточка выделена через `featured`. При нажатии `handlePress` сохраняет страну, и над списком появляется её название и столица. Список лежит в `ScrollView`, потому что `FlatList` по методичке вводится в Sprint 2.

Путь нажатия: `Pressable` → `onPress(topic)` → `handlePress` → `setSelected` → экран перерисовывается.

## Metro, Hermes и Fast Refresh

- **Metro** собирает код проекта и отдаёт его на телефон или в браузер.
- **Hermes** выполняет этот код на устройстве. В `app.json` движок не указан, поэтому используется Hermes по умолчанию. Это видно в адресе бандла (`transform.engine=hermes`), а сборка под Android даёт файл `.hbc` — байт-код Hermes.
- **Fast Refresh** проверен: после изменения `title` в `src/data/topics.ts` экран обновился без перезапуска.

## Проверка

| Проверка                                            | Результат                          |
|-----------------------------------------------------|------------------------------------|
| <code>npm run typecheck</code>                      | без ошибок                         |
| <code>npx expo-doctor</code>                        | 21/21                              |
| <code>any</code> в коде                             | нет                                |
| Установка с нуля ( <code>npm ci</code> )            | 488 пакетов, без ошибок            |
| Сборка под Android ( <code>npx expo export</code> ) | бандл собран                       |
| Экран                                               | 12 карточек                        |
| Нажатие на Australia                                | над списком «Australia · Canberra» |
| Canada без картинки                                 | заглушка <b>CAN</b>                |
| Fast Refresh                                        | работает                           |

## Как показать на приёмке

| Шаг                      | Что сделать                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------|
| Запуск                   | <code>npm install</code> , затем <code>npm start</code>                                   |
| Три и больше карточек    | на экране 12 карточек                                                                     |
| Fast Refresh             | изменить <code>title</code> в <code>src/data/topics.ts</code>                             |
| Ошибка TypeScript        | убрать <code>onPress={handlePress}</code> — <code>npm run typecheck</code> покажет ошибку |
| Карточка не знает данных | открыть импорты <code>TopicCard.tsx</code>                                                |

| Шаг                          | Что сделать                                     |
|------------------------------|-------------------------------------------------|
| Путь нажатия                 | Pressable → onPress → handlePress → setSelected |
| Pull request, CHANGELOG, тег | PR #2, запись [sprint-1], тег sprint-1          |

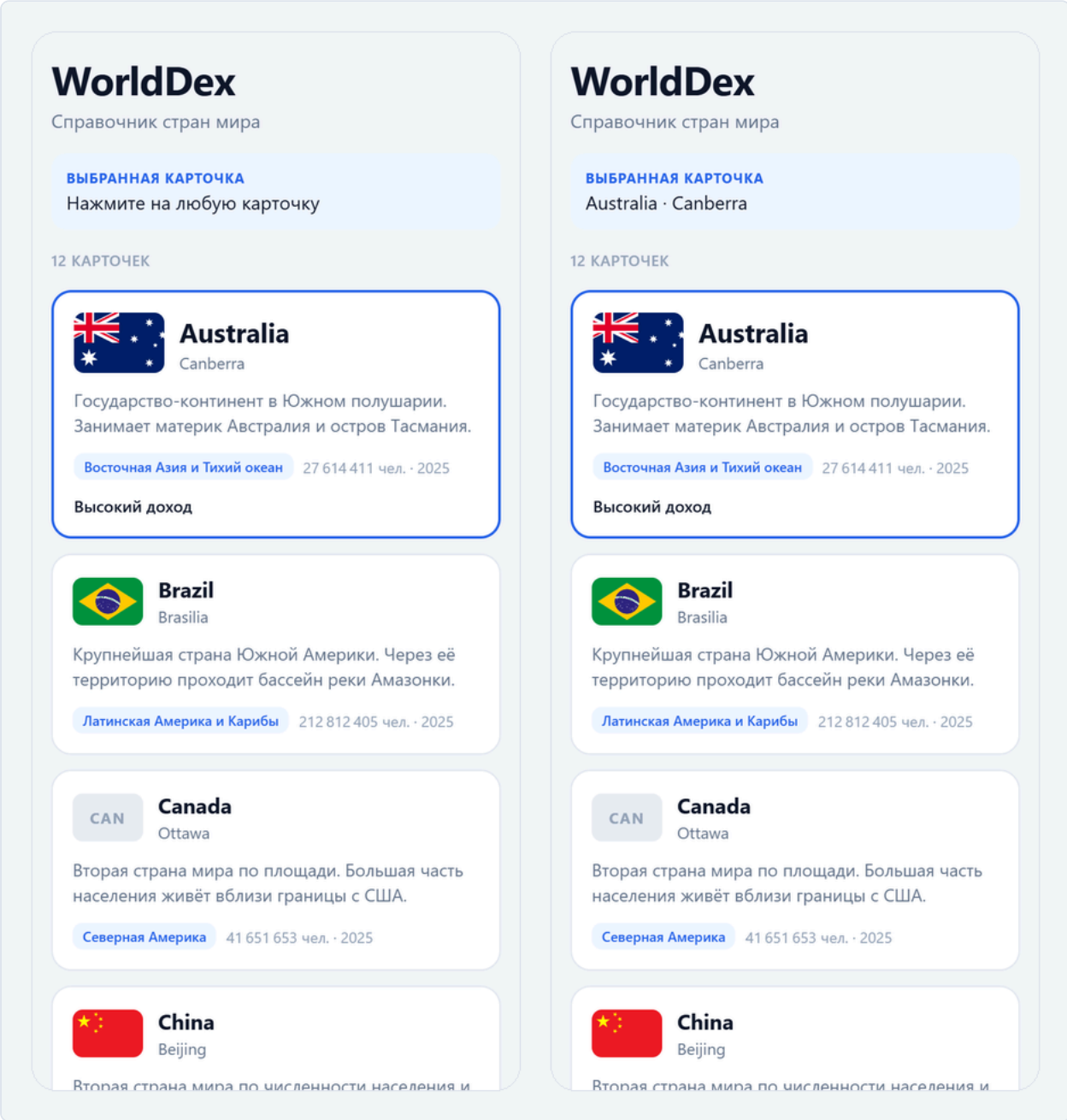


Рисунок 1 — Каталог при запуске (слева) и после нажатия на Australia (справа)

## 03 / РЕЗУЛЬТАТЫ

### Результат

Первый экран готов: 12 карточек стран, все нарисованы одним компонентом. Данные лежат отдельно и проверяются типами, нажатие работает, карточка без картинки не ломает вёрстку.

Таблица 1 — Критерии приёмки

| Критерий                                                     | Как выполнен                                                      | Статус    |
|--------------------------------------------------------------|-------------------------------------------------------------------|-----------|
| Проект запускается после установки по README                 | <code>npm install, npm start;</code><br>проверено на чистом клоне | выполнено |
| Данные соответствуют типу Topic, id уникальные               | <code>TOPICS: Topic[]</code> , 12 кодов стран без повторов        | выполнено |
| TopicCard используется не меньше трёх раз и не хранит данные | 12 карточек через <code>map</code> , данных в компоненте нет      | выполнено |
| Props и callback типизированы без <code>any</code>           | <code>TopicCardProps</code> , <code>any</code> нет                | выполнено |
| Отсутствие картинки обрабатывается                           | заглушка с кодом страны                                           | выполнено |
| Нажатие вызывает callback с id или Topic                     | <code>onPress(topic)</code>                                       | выполнено |
| Fast Refresh проверен, понятна разница Metro и Hermes        | проверено, описано выше                                           | выполнено |
| Sprint 0 сохранён                                            | <code>docs/</code> и история репозитория на месте                 | выполнено |

Таблица 2 — Ограничения

| Ограничение                                       | Статус    |
|---------------------------------------------------|-----------|
| Не использовать <code>any</code> без обоснования  | соблюдено |
| Не хранить данные внутри TopicCard                | соблюдено |
| Не копировать разметку карточки                   | соблюдено |
| Не подключать API, Redux, базу данных и навигацию | соблюдено |
| Не использовать готовые библиотеки карточек       | соблюдено |

Таблица 3 — Definition of Done

| Пункт                                                      | Статус                                                              |
|------------------------------------------------------------|---------------------------------------------------------------------|
| Сценарий работает на целевой платформе                     | проверено в браузере и сборкой под Android; на телефоне — на защите |
| Проверка типов и проверки качества проходят                | выполнено                                                           |
| В README есть команды запуска и структура проекта          | выполнено                                                           |
| Изменения прошли через pull request                        | выполнено, PR #2                                                    |
| В CHANGELOG есть запись о витрине                          | выполнено                                                           |
| Есть тег <code>sprint-1</code> и скриншот                  | выполнено                                                           |
| Можно объяснить компонент, props, callback, Metro и Hermes | выполнено                                                           |

## Проблемы и решения

**1. Backlog не совпал с методичкой.** В backlog первым спринтом стояла загрузка списка из API, а методичка запрещает API в первых двух спринтах. Загрузка перенесена на Sprint 3, для текущих задач добавлены истории US-10 и US-11.

**2. FlatList нельзя до Sprint 2.** 12 карточек не помещаются на экран, поэтому список завернут в `ScrollView`.

**3. Флаги появляются не сразу.** Картинки грузятся из интернета, и сначала на их месте было пусто. У `Image` задан цвет фона, теперь место под флагом видно сразу.

**4. Тег поставлен раньше времени.** Тег `sprint-1` был создан до слияния pull request и указывал на коммит Sprint 0. Тег удалён и поставлен заново после слияния.

**5. OneDrive мешает git.** При переключении веток git не смог удалить папку `src/components`, потому что её держал OneDrive. Помогает пауза синхронизации.

## Ретроспектива

- **Получилось:** карточку можно переиспользовать без изменений.
- **Мешало:** расхождение backlog с методичкой и ошибки при публикации.
- **Долг:** данные лежат в коде; `ScrollView` рисует все карточки сразу.

- **Дальше:** `FlatList`, поиск, фильтр и выбор карточек.
- 

## 04 / ЗАВЕРШЕНИЕ

### Выводы

Сделан первый экран справочника из типизированных компонентов. Карточка получает данные через `props` и не знает, откуда они взялись, поэтому в следующем спринте её не пришлось переделывать. Все критерии приёмки выполнены.

---

## 05 / КОНТРОЛЬНЫЕ ВОПРОСЫ

### Контрольный вопрос 1

**Чем компонент отличается от экрана приложения?**

Компонент — отдельная часть интерфейса, которую можно использовать много раз. `TopicCard` рисует одну страну и ничего не знает об остальном приложении. Экран — то, что пользователь видит целиком. `CatalogScreen` хранит состояние и собирает интерфейс из компонентов.

### Контрольный вопрос 2

**Зачем данные передаются в `TopicCard` через `props`?**

Чтобы карточка не зависела от конкретных данных. Один компонент рисует все 12 стран, получая каждый раз другой объект. Если данные начнут приходить из API, карточку менять не придётся.

### Контрольный вопрос 3

**Почему `fixtures` не должны находиться внутри карточки?**

Тогда карточка сможет показывать только эти данные и её нельзя будет переиспользовать. Хранение данных и их отображение — разные задачи, поэтому данные лежат в `src/data/topics.ts`.

### Контрольный вопрос 4

**Чем обязательный `prop` отличается от `optional prop`?**

Обязательный prop нужно передавать всегда, иначе TypeScript покажет ошибку. У `TopicCard` обязательны `topic` и `onPress`. Необязательный prop помечается `?` и может отсутствовать, поэтому у `featured` есть значение по умолчанию `false`.

## Контрольный вопрос 5

**Что TypeScript проверяет до запуска приложения, а что он не проверяет?**

До запуска TypeScript проверяет типы: переданы ли нужные props, правильные ли поля в данных, подходит ли функция под тип callback. Он не проверяет то, что происходит во время работы: загрузилась ли картинка, какие данные пришли из сети, правильно ли работает логика. Metro при сборке просто удаляет типы, поэтому проверять их нужно отдельно командой `npm run typecheck`.

## Контрольный вопрос 6

**Какую задачу выполняет Metro?**

Metro — сборщик. Он собирает файлы проекта в один бандл, отдаёт его на устройство и пересобирает при изменениях. На этом работает Fast Refresh.

## Контрольный вопрос 7

**Какую задачу выполняет Hermes?**

Hermes — движок JavaScript на устройстве. Он выполняет бандл, который заранее переведён в байт-код, поэтому приложение быстрее запускается и тратит меньше памяти.

## Источники

1. React Native. Core Components and APIs.  
<https://reactnative.dev/docs/components-and-apis>
2. React Native. Image. <https://reactnative.dev/docs/image>
3. React Native. Pressable. <https://reactnative.dev/docs/pressable>
4. React Native. Using TypeScript. <https://reactnative.dev/docs/typescript>
5. React Native. Fast Refresh. <https://reactnative.dev/docs/fast-refresh>

6. React Native. Using Hermes. <https://reactnative.dev/docs/hermes>
7. Metro. Concepts. <https://metrobundle.dev/docs/concepts>
8. React. Passing Props to a Component. <https://react.dev/learn/passing-props-to-a-component>

Дата обращения ко всем источникам: 16.09.2026.